

1 RISC-V Instructions

1.1 Assume we have an array in memory that contains `int *arr = {1,2,3,4,5,6,0}`. Let register `s0` hold the address of the element at index 0 in `arr`. You may assume integers are four bytes and our values are word-aligned. What do the following snippets of RISC-V code do? Assume that all the instructions are run one after the other in the same context.

(a) `lw t0, 12(s0)`

Sets `t0` equal to `arr[3]`

(b) `sw t0 16(s0)`

Stores `t0` into `arr[4]`

(c) `slli t1, t0, 2`
`add t2, s0, t1`
`lw t3, 0(t2)`
`addi t3, t3, 1`
`sw t3, 0(t2)`

Increments `arr[4]` by 1.

1st line sets `t1 = 16`

2nd line adds it to `s0` so that it now points at `arr[4]`

3rd-5th line loads the value at `arr[4]`, increments by one, and stores it back

(d) `lw t0, 0(s0)`
`xori t0, t0, 0xFFF`
`addi t0, t0, 1`

Sets `t0` to `-1 * arr[0]`

2 Lost in Translation

2.1 Translate the code verbatim between C and RISC-V. The comments above the code indicate which registers to store the variables.

C	RISC-V
<pre>// s0 -> a // s1 -> b // s2 -> c // s3 -> z int a = 4, b = 5, c = 6; int z = a + b + c + 10;</pre>	<pre>addi s0, x0, 4 addi s1, x0, 5 addi s2, x0, 6 add s3, s0, s1 add s3, s3, s2 addi s3, s3, 10</pre>
<pre>// int *p = intArr; // s0 -> p; // s1 -> a; *p = 0; int a = 2; p[1] = p[a] = a;</pre>	<pre>sw x0, 0(s0) addi s1, x0, 2 sw s1, 4(s0) slli t0, s1, 2 add t0, t0, s0 sw s1, 0(t0)</pre>
<pre>// s0 -> a, // s1 -> b int a = 5; int b = 10; if (a + a == b) { a = 0; } else { b = a - 1; }</pre>	<pre>start: addi s0, x0, 5 addi s1, x0, 10 add t0, s0, s0 bne t0, s1, else add s0, x0, x0 jal x0, exit else: addi s1, s0, -1 exit: ...</pre>
<pre>// Compute s1 = 2^30 int s0 = 0; int s1 = 1; for (; s0 != 30; s0 += 1) { s1 *= 2; }</pre>	<pre>start: addi s0, x0, 0 addi s1, x0, 1 addi t0, x0, 30 loop: beq s0, t0, exit slli s1, s1, 1 addi s0, s0, 1 jal x0, loop exit: ...</pre>

C	RISC-V
<pre>// s0 -> n // s1 -> sum for (int sum = 0; n > 0; n--) { sum += n; }</pre>	<pre>start: addi s1, x0, 0 loop: beq s0, x0, exit add s1, s1, s0 addi s0, s0, -1 jal x0, loop exit: ...</pre>

3 RISC-V Memory Access

For Q3.1 – Q3.2, use the instructions and memory to figure out what the code does. Recall that RISC-V is little-endian and byte addressable. For any unknown instructions, use the [CS 61C reference card!](#)

- 3.1 Fill in the registers with the values they contain after the code finishes executing.

```
li t0 0x00FF0000
lw t1 0(t0)
addi t0 t0 4
lh t2 2(t0)
lw s0 0(t1)
lb s1 3(t2)
```

t0	0x00FF0004	0xFFFFFFFF	
t1	36	0x00FF0004	0x000C561C
t2	0x0000000C	0x00FF0000	36
s0	0xDEADB33F	0x00000036	0xFDFDFDFD
s1	0xFFFFFC5	0x00000024	0xDEADB33F
		0x0000000C	0xC5161C00
		0x00000000	

- **t0**: Line 3 adds 4 to the initial address.
- **t1**: Line 2 loads the 4-byte word from address 0x00FF0000.
- **t2**: Line 4 loads two bytes starting at the address $0x00FF0004 + 2 = 0x00FF0006$. This returns 0x000C, sign-extended to 0x0000000C.
- **s0**: Line 5 loads the word starting at address $36 = 0x24$ which is 0xDEADB33F.
- **s1**: Line 6 loads the MSB starting of the 4-byte word at address 0xC. The value is 0xC5 which is sign-extended to 0xFFFFFC5.

- 3.2 Fill in the memory diagram and `t3` register with the values contained in them after the code finishes executing. The values in the `t0`, `t1`, and `t2` registers at the start of program execution have been provided to you. Assume that all memory starts out initialized to zeros.

<code>sw t0 0(t1)</code>	<code>t0</code>	<code>0xABADCAF8</code>	<code>0xFFFFFFFF</code>	<code>0x00000000</code>
<code>addi t0 t0 4</code>				...
<code>sh t1 2(t0)</code>	<code>t1</code>	<code>0xF0120504</code>	<code>0xF0120504</code>	<code>0xABADCAF8</code>
<code>sh t2 0(t0)</code>				...
<code>lw t3 0(t1)</code>	<code>t2</code>	<code>0xBEEFDAB0</code>	<code>0xBEEFDAB0</code>	<code>0x00000000</code>
<code>sb t1 1(t3)</code>				...
<code>sb t2 3(t3)</code>	<code>t3</code>	<code>0xABADCAF8</code>	<code>0xABADCAFC</code>	<code>0x0504DAB0</code>
			<code>0xABADCAF8</code>	<code>0xB0000400</code>
				...
			<code>0x00000000</code>	<code>0x00000000</code>

4 RISC-V Calling Convention

- 4.1 Consider the following blocks of code:

<pre> main: # Prologue # Saves ra # Code omitted addi s0 x0 5 # Breakpoint 1 jal ra foo # Breakpoint 3 mul a0 a0 s0 # Code omitted # Epilogue # Restores ra j exit </pre>	<pre> foo: # Prologue # Saves s0 # Code Omitted addi s0 x0 4 # Breakpoint 2 # Epilogue # Restores s0 jr ra </pre>
---	---

- a) Does `main` always behave as expected, as long as `foo` follows calling convention?

Yes, since `foo` saves the saved registers and `main` saves the return address.

- b) What does `s0` store at breakpoint 1? Breakpoint 2? Breakpoint 3?

Breakpoint 1: 5, Breakpoint 2: 4, Breakpoint 3: 5

- c) Now suppose that `foo` didn't have a prologue or epilogue. What would `s0` store at each of the breakpoints? Would this cause errors in our code?

Breakpoint 1: 5, Breakpoint 2: 4, Breakpoint 3: 4. This would cause errors because we rely on `s0` having the value of 5 in our calculations in `main`.

In part (c) above, we see one way how not following calling convention could make our code misbehave. Other things to watch out for are: assuming that `a` or `t` registers will be the same after calling a function, and forgetting to save `ra` before calling a function.

4.2 Function `myfunc` takes in two arguments: `a0`, `a1`. The return value is stored in `a0`. In `myfunc`, `generate_random` is called. It takes in 0 arguments and stores its return value in `a0`.

```
myfunc:
# Prologue (omitted)

addi t0 x0 1
slli t1 t0 2
add t1 a0 t1
add s0 a1 x0

jal generate_random

add t1 t1 a0
add a0 t1 s0

# Epilogue (omitted)
ret
```

a) Which registers, if any, need to be saved on the stack in the prologue?

Answer: `s0`, `ra`

We must save all `s`-registers we modify. In addition, if a function contains a function call, register `ra` will be overwritten when the function is called (i.e. `jal ra label`). `ra` must be saved before a function call. It is conventional to store `ra` in the prologue (rather than just before calling a function) if the function contains a function call. `myfunc` contains the function call `generate_random`.

b) Which registers, if any, need to be saved on the stack before calling `generate_random`?

Answer: `t1`

Under calling conventions, all the `t`-registers and `a`-registers may be changed by `generate_random`, so we must store all of these which we need to know the value of after the call. A total of 2 `t`-registers are used before calling `generate_random`, `t0` and `t1`, but only `t1`'s value is referenced again after the function call.

c) Which registers, if any, need to be restored from the stack in the epilogue before returning?

Answer: `s0`, `ra`

This mirrors what we did in the prologue.

5 Recursive Calling Convention

Write a function **sum_squares** in RISC-V that, when given an integer **n** and a constant **m**, returns the summation below. If **n** is not positive, then the function returns **m**.

$$m + n^2 + (n - 1)^2 + (n - 2)^2 + \dots + 1^2$$

To implement this, we will use a tail-recursive algorithm that uses the **a1** register to help with recursion.

sum_squares: Return the value $m + n^2 + (n - 1)^2 + \dots + 1^2$		
Arguments	a0	A 32-bit number n . You may assume $n \leq 10000$.
	a1	A 32-bit number m .
Return value	a0	$m + n^2 + (n - 1)^2 + (n - 2)^2 + \dots + 1^2$. If $n \leq 0$, return m

For this problem, you are given a RISC-V function called **square** that takes in a single integer and returns its square.

square: Squares a number		
Arguments	a0	n
Return value	a0	n^2

5.1 Since this a recursive function, let's implement the base case of our recursion:

```
sum_squares:
    bge x0 a0 zero_case

    # To be implemented in the next question

zero_case:
    mv a0 a1
    jr ra
```

5.2 Next, implement the recursive logic. *Hint: if you let $m' = m + n^2$, then*

$$m + n^2 + (n - 1)^2 + \dots + 1^2 = m' + (n - 1)^2 + \dots + 1^2$$

```

sum_squares:
# Handle zero case (previous question)
bge x0 a0 zero_case

mv t0 a0
jal ra square

add a1 a0 a1
addi a0 t0 -1

jal ra sum_squares
jr ra

zero_case:
# Handle zero case (previous question)
jr ra

```

- 5.3 Now, think about calling convention from the caller perspective. After the call to **square**, what is in **a0** and **a1**? Which one of the registers will cause a calling convention violation?

a0 will contain n^2 , and **a1** will contain garbage data, causing a calling convention violation. The register **t0** will also hold garbage, which would also cause a calling convention violation.

- 5.4 What about the recursive call? What will be in **a0** and **a1** after the call to **sum_squares**?

a0 will contain $m + n^2 + \dots + 1^2$ since the final call to **jal ra sum_squares** will perform a **mv a0 a1** operation. Since **a0** now contains the expected return value, we no longer care about the value in **a1**, and can directly return.

According to calling conventions, the value in **a1** after the call is not guaranteed and should be considered unreliable. However, in this specific implementation, **a1** happens to also contain $m + n^2 + \dots + 1^2$. It is the job of whichever function called **sum_squares** to deal with saving caller-saved registers if they are needed in the future.

- 5.5 Now, go back and fix the calling convention issues you identified. Note that not all blank lines may be used. There may also be another caller saved register that you need to save as well!

```

sum_squares:
# Handle zero case (previous question)
mv t0 a0

# Save caller saved registers on the stack
addi sp sp -12
sw a1 0(sp)
sw t0 4(sp)
sw ra 8(sp)
jal ra square
# Restore register and stack
lw a1 0(sp)
lw t0 4(sp)
lw ra 8(sp)
addi sp sp 12

add a1 a0 a1
addi a0 t0 -1

# Save caller saved registers on the stack
# Note that we don't need to save a1 and t0
# because we do not need their values
# after the function call.
addi sp sp -4
sw ra 0(sp)
jal ra sum_squares
# Restore caller saved registers on the stack
lw 0(sp)
addi sp sp 4

jr ra

zero_case:
# Handle zero case (previous question)
jr ra

```

- 5.6 Now, from a callee perspective, do we have to save any registers in the prologue and epilogue? If yes, what registers do we have to save, and where do we place the prologue and epilogue? If no, briefly explain why.

No, we do not have to take callee saved registers into account because we do not use any callee saved registers. However, since we call two functions, it is possible to save **ra** in the prologue and restore it in an epilogue immediately before the **jr ra** before the **zero_case** label.